ProbOS — Month 1, Week 4
DP Profiling, C++/OpenMP, PDSL v0.1, Cross-Discipline Examples

Nisong Monyimba

July 3, 2026

Week 4 goal

Three parallel threads: performance (honest DP profiling, C++/OpenMP kernel), language (PDSL compiler v0.1), and breadth (three cross-discipline examples proving the kernel generalises beyond batteries). Close the month with a retrospective and Month 2 plan.

Day-by-day plan

Monday – DP optimisation profiling

- Profile candidate optimisations honestly: cache SALib's `_build_problem()` at `__init__` (measured: 1040x), CLT convergence subset trick (measured: 45x)
- Also test `inv_RT` precompute – measure it, and report the true result (˜1x, no real speedup) rather than assuming or claiming a win

Tuesday – C++ BatteryCell + OpenMP

- Port `BatteryModel2Cell` to C++20: `cpp/include/kernel/battery_cell.hpp`
- `MonteCarloEngineOMP`: OpenMP-parallel particle propagation
- Benchmark against Python: measured 7x speedup (serial C++ vs Python, not yet parallel-vs-Python)

Wednesday – PDSL compiler v0.1

- Grammar (Lark) → AST → codegen → compiler pipeline
- `python/pdsl/`: `grammar.lark`, `ast_nodes.py`, `parser.py`, `codegen.py`, `compiler.py`
- Literature update: added modern citations (Feng 2018, Saltelli 2019, Chopin & Papaspiliopoulos 2020, Naesseth 2019, FDA 2019) to the manuscript

Thursday – v0.1.0 checkpoint decision

- Decision: defer formal publication/Zenodo DOI/GitHub Release to Year 2 – a git tag is version-control hygiene, a Zenodo DOI is publication infrastructure with zero value before there is a paper citing it
- Repo hygiene: moved all generated PNGs into `outputs/figures/`, fixed resulting CI break (missing `lark` dependency)
- `check_ci.sh`: permanent CI-watching script

Friday/Saturday – Three cross-discipline examples

- `week4_option_pricer.py` – European call option (Black-Scholes-Merton GBM), validated against the closed-form price
- `week4_ed_queue.py` – M/M/1 emergency department queue, validated against closed-form queueing theory (ρ, L, W)
- `week4_clinical_trial.py` – adaptive Bayesian trial (Beta-Binomial), informed by FDA (2019) adaptive design guidance
- Each needed zero kernel changes – only a new `Model` subclass, proving domain-agnosticism rather than assuming it

Sunday – Month 1 retrospective + Month 2 plan

- Wrote `docs/retrospectives/month1_retrospective.md` and `month2_plan.md`
- Compiled both to PDF under `docs/monthly_plans/month1/` and `docs/monthly_plans/month2/`
- Created `docs/monthly_plans/overall/main.tex`: replaced the earlier inconsistent "Year 1/2/3" labelling (which did not align with actual calendar months) with an honest Month 1–24 numbering, most of it explicitly DIRECTIONAL rather than over-specified

What was actually accomplished (retrospective)

- DP profiling: two genuine wins (1040x, 45x) and one honest non-win (`inv_RT` ˜1x) – documented rather than glossed over
- C++ `BatteryCell` + `MonteCarloEngineOMP`: 7x serial speedup over Python confirmed by measurement
- PDSL compiler v0.1: full grammar → AST → codegen → compiler pipeline, 28 tests, including the tricky unary-minus-in- function-call parser case
- Three cross-discipline examples, each validated against a closed-form or literature benchmark, not just "runs without crashing"
- `check_ci.sh` established as a permanent, repeatable CI verification step
- Month 1 retrospective and Month 2 plan written and compiled to PDF, plus the overall 24-month roadmap correcting earlier inconsistent year labelling

- *Later, before Week 7:* a full pre-week quality audit system was added retroactively – `docs/standards/quality_s` and `pre_week_audit.sh` – covering coverage, property-based testing (Hypothesis), security scanning (bandit, pip-audit), and packaging/build verification. This audit caught and fixed real latent bugs from Week 4's own work: `python/examples/` missing `__init__.py`, 4 mypy errors that had never been checked by CI, a stale coverage threshold, and a bandit finding on the PDSL compiler's `exec()` call (verified as a sound, documented security boundary rather than a real vulnerability)

Numbers at Week 4 close (updated through the pre-Week-7 audit)

- Python tests: 322/322 passing
- C++ tests: 13/13 passing
- mypy strict: 0 errors (full `python/` tree)
- ruff: 0 warnings (full `python/` tree)
- Coverage: 90.79% (floor: 85%)
- bandit: 0 unresolved findings
- pip-audit: 0 known CVEs
- `pre_week_audit.sh`: 21/21 checks passed

Carried into Month 2

Month 1 closes with a validated forward-simulation kernel across four domains (battery physics, finance, hospital operations, clinical medicine), a working compiler front-end, and – critically – a standing quality-audit protocol now running automatically in CI before every future week's work begins. Month 2 turns to sequential inference (`ParticleFilter`), binding the C++ kernel into the Python package (pybind11), and eventually a FastAPI service layer.